MedScan AI – End-to-End Project Workflow

---

1. Model Training (Google Colab)

Skin Cancer Detection (PyTorch)

Dataset
HAM10000 (Kaggle)

Architecture
Custom SkinCancerModel built on top of **timm backbones** with:

- GeM Pooling
- Linear (512)
- BatchNorm
- SiLU activation head

Models Trained

- EfficientNetV2M
- EfficientNetV2S
- ConvNeXt-Base

Ensemble Strategy

Final prediction is generated by **averaging softmax probabilities across all three models**.

Image Transforms (Albumentations)

- Resize (300, 300)
- Normalize (ImageNet statistics)
- ToTensorV2

Class Order

Alphabetically sorted lowercase dx codes:

akiec
bcc
bkl
df
mel
nv
vasc

Saved Model Files

model1_efficientnetv2m.pth
model2_efficientnetv2s.pth
model3_convnext.pth

Each checkpoint stores:

model_state_dict

Performance

AUROC: **0.9609**

Pneumonia Detection (Keras)

Dataset

Chest X-Ray Pneumonia Dataset (Kaggle)

Architecture

Custom **Sequential CNN**

Input

224 × 224 RGB images

Normalization

image / 255

Output Layer

Sigmoid activation for **binary classification**

Saved Model

pneumodel.h5

Diabetes Prediction (Keras)

Dataset

Pima Indians Diabetes Dataset
(Kaggle: uciml/pima-indians-diabetes-database)

Model Architecture

Dense(256) → BatchNorm → ReLU → Dropout(0.3)
Dense(128) → BatchNorm → ReLU → Dropout(0.3)
Dense(64) → BatchNorm → ReLU → Dropout(0.2)
Dense(32) → ReLU
Dense(1) → Sigmoid

Data Preprocessing

The following columns contained invalid **zero values** and were replaced with NaN:

- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI

Missing values were then filled using the **median of each column**.

After cleaning, **StandardScaler** was applied.

Scaler Parameters Saved As

diabetes_scaler.json

Contains:

- mean array
- scale array

Model Weights Saved As

diabetes_weights.h5

Important Deployment Note

Instead of saving the full model:

```
model.save()
```

only weights were saved:

```
model.save_weights()
```

This prevents **Keras version incompatibility errors**, such as:

quantization_config deserialization errors

when deploying across different environments.

---

2. Upload Models to HuggingFace Model Repositories

Three separate **model repositories** were created.

Skin Cancer Models

SaswatML123/Skin_cancer_detection

Contains:

model1_efficientnetv2m.pth
model2_efficientnetv2s.pth
model3_convnext.pth

Pneumonia Model

SaswatML123/PneuModel

Contains:

pneumodel.h5

Diabetes Model

SaswatML123/DiabetesModel

Contains:

diabetes_model.h5
diabetes_scaler.json

These repositories act as **model storage only**.
No inference widget is used.

---

3. FastAPI Backend Development

The backend was implemented using **FastAPI**.

API Endpoints

Health Check

GET /health

Used for monitoring and keep-alive.

Root Endpoint

GET /

Returns a list of available endpoints.

Skin Cancer Prediction

POST /predict/skin

Accepts an **image upload**.

Returns predicted class probabilities.

Pneumonia Prediction

POST /predict/pneumonia

Accepts **chest X-ray image upload**.

Returns pneumonia probability.

Diabetes Prediction

POST /predict/diabetes

Accepts a **JSON body containing 8 patient parameters**.

Example:


```
{
 "Pregnancies": 2,
 "Glucose": 130,
 "BloodPressure": 70,
 "SkinThickness": 25,
 "Insulin": 80,
 "BMI": 28.5,
 "DiabetesPedigreeFunction": 0.35,
 "Age": 32
}
```

Model Loader System

Implemented in:

model_loader.py

Responsibilities:

- Download model weights from HuggingFace
- Cache models locally
- Rebuild model architectures before loading weights

Local cache location:

/app/model_cache/

Downloads occur **only once**, then cached.

Skin Cancer Model Loading

The backend recreates the exact training architecture:

SkinCancerModel

including:

- GeM pooling
- Linear 512 head
- BatchNorm
- SiLU activation

Weights loaded using:

model_state_dict

Pneumonia Model Loading

Loaded directly using:

tf.keras.models.load_model()

Diabetes Model Loading

Architecture rebuilt using:

_build_diabetes_model()

Weights loaded using:

model.load_weights()

This avoids **Keras version mismatches**.

---

4. Deployment as HuggingFace Docker Space

Backend deployed as a **Docker Space**.

Repository:

SaswatML123/MedScan-API

Files in the Space

Dockerfile
README.md
requirements.txt
main.py
model_loader.py

Docker Configuration

Base image:

python:3.10-slim

Exposed port:

7860

Server started with:

python main.py

Important Deployment Design

Models are **NOT stored inside the Space**.

Instead:

1. Backend downloads models from HuggingFace repos
2. Models are cached locally in /app/model_cache/
3. Subsequent requests reuse cached models.

CORS Configuration

allow_origins = ["*"]

This allows the **HTML frontend** to call the backend API.

---

5. HTML Frontend Development

Frontend built using **plain HTML, CSS, and JavaScript**.

No framework was used.

Pages

index.html

Landing page displaying module cards.

---

skin.html

- Upload skin lesion image
- Displays probability bars for predictions.

---

pneumonia.html

- Upload chest X-ray
- Displays pneumonia prediction.

---

diabetes.html

- Form with **8 clinical input fields**
- Displays prediction probabilities.

Backend Communication

All pages call the API:

https://saswatML123-medscan-api.hf.space

UI Behavior

- Loading animation while waiting for predictions
- Probability visualization bars
- Error handling for invalid input.

---

6. Keep-Alive System

HuggingFace **free Spaces automatically sleep** after inactivity.

Two keep-alive layers were implemented.

Frontend Keep-Alive

JavaScript periodically calls:

/health

every **10 minutes** using setInterval().

External Keep-Alive

External service:

cron-job.org

also pings:

/health

every **10 minutes**.

This ensures the backend **remains active**.

---

7. Debugging Skin Cancer Predictions

Initial predictions were incorrect.
Several issues were identified and fixed.

Issue 1: Incorrect Image Size

Backend used:

224 × 224

Training used:

300 × 300

Fixed by updating backend preprocessing.

Issue 2: Incorrect Image Transforms

Backend used:

torchvision.transforms

Training used:

albumentations

Solution: switch backend preprocessing to **Albumentations**.

Issue 3: Incorrect Model Architecture

Backend initially used:

timm.create_model()

with the default classification head.

However, the trained model used a **custom GeM pooling head**.

Solution: recreate the full SkinCancerModel architecture.

Issue 4: Cached Docker Container

After uploading updates, the Space continued running old code.

Solution:

Perform a **Factory Reboot** in Space settings.

Project Structure

MediScan/

```
frontend/
├── index.html
├── skin.html
├── pneumonia.html
└── diabetes.html

space/
├── main.py
├── model_loader.py
├── requirements.txt
├── Dockerfile
└── README.md
```